

PATHFINDER website — documentation

Every guide bundled into one printable reference

Documentation index

Everything you need to run and update the PATHFINDER project website.

Find your role

- I'm a partner submitting news or an event → [composer.md](#)
- I'm the publisher committing content to the site → [publishing.md](#)
- I want to edit partner descriptions, team bios, or the About page → [editing-directly.md](#)
- I need to add or swap an image → [managing-images.md](#)
- Something broke → [troubleshooting.md](#)
- I'm the technical maintainer deploying or configuring the site → [deployment.md](#)

About this site in one paragraph

Static site built with [Astro](#), styled with [Tailwind CSS](#), deployed automatically to GitHub Pages on every push to `main`. All content is plain text files — news and events as Markdown files, partners and the Advisory Board as Markdown too. There is no database, no admin login for the public, and no backend. Only repository collaborators can push changes, and (with branch protection) every change goes through a reviewed pull request.

The two ways content gets added

1. **Composer (recommended for coordinators)** — a form at `/admin/compose` on the live site. Fill in the form, download a `.md` file, send it to the publisher. Full guide: [composer.md](#).
2. **Directly on GitHub** — edit or create Markdown files in `src/content/`. Recommended for the publisher and anyone comfortable with GitHub's web editor. Full guide: [editing-directly.md](#).

Both routes end up in the same place: a Markdown file in `src/content/news/` or `src/content/events/`. The build reads those files and generates the website.

The site's map (for reference)

Section	URL	Source
Home	/	src/pages/index.astro
About	/about	src/pages/about.astro
Consortium (partners + Advisory Board)	/consortium	src/pages/consortium.astro + src/content/partners/*.md + src/content/advisory-board/*.md
News & events	/news-events	src/pages/news-events/index.astro + src/content/news/*.md + src/content/events/*.md
Outputs	/outputs	src/pages/outputs/index.astro + src/content/outputs/*.md
Composer (unlisted)	/admin/compose	src/pages/admin/compose.astro

Using the Composer

The Composer is a form on the PATHFINDER website that lets you write a news item or an event and download it as a ready-to-publish file — **without touching GitHub, YAML, or code**.

URL: `https://<your-live-url>/admin/compose`

The page is unlisted (not in the site's menu, hidden from search engines). Bookmark the URL when your maintainer sends it to you.

What the Composer does — and doesn't

-  Takes what you type and formats it correctly for the website.
-  Shows a live preview so you can see how the news card and article will look.
-  Generates the right filename and downloads the file for you.
-  Does **not** publish anything on its own. You still need to send the downloaded file to your publisher.

Think of it as a "clean the format for me" tool. Publishing is a separate step handled by the designated publisher (contact your project coordinator to know who that is).

Writing a news item

1. Go to `/admin/compose`.
2. Confirm the toggle at the top says **News item** (the darker button). If not, click it.
3. Fill in the fields:
 - **Title** — a short, punchy headline (max 120 characters).
 - **Date** — the date of publication. Defaults to today.
 - **Summary** — one or two sentences that will show on the News card. This is what people see before clicking through.
 - **Body** — the full article. Uses Markdown for formatting (see below).
 - **Tags (optional)** — one or two labels, comma-separated (e.g. `event-recap, sarajevo`).
 - **Author (optional)** — your name or the name of whoever wrote the piece.
4. **(Optional)** Click **Add an image** to include a hero photo. See [managing-images.md](#) for what to put here.
5. Watch the **Preview** on the right — the news card and article both update as you type.
6. When you're happy, click  **Download .md**. A file like `2027-05-15-my-title.md` saves to your Downloads folder.

7. Send that file to the publisher (see the "Sending to the publisher" section below).

Writing an event

Same flow as news, but click the **Event** toggle at the top first. The form changes:

- **Title** — the event's name (e.g. "Sofia Forum on AI and the Future of Work").
- **Start date** and **End date (optional)** — the date(s) the event runs.
- **Location** — city and country (e.g. "Sofia, Bulgaria" or "Online").
- **Country** — pick from the dropdown. "Online" is a valid choice for virtual events.
- **Format** — In-person / Online / Hybrid.
- **Summary** — a one- or two-sentence description shown on the events grid.
- **Body** — full agenda, background, or details in Markdown.
- **Registration URL (optional)** — a link where people can register.

Everything else works the same. Download, send to publisher.

Formatting the body

There are two ways to format text in the **Body** field:

1. Toolbar buttons (easiest)

Right above the body textarea there's a small toolbar:

- **B** — bold. Select some text and click B, or click B and start typing.
-  **Link** — insert a link. Select the text you want linked, click Link, then type or paste the URL in the popup.
- **H2** — turn the current line into a section heading.
- **H3** — turn the current line into a sub-heading.
- **• List** — turn the current line (or selected lines) into a bulleted list.
- **“ Quote** — turn the current line into a quote block.

The preview on the right updates immediately so you see exactly what will render.

2. Type Markdown directly

The toolbar is just a shortcut — you can type the same syntax by hand. That's plain text with a few simple rules:

```
## A section heading
### A smaller heading

A paragraph. Just type normally.

Leave a blank line between paragraphs.

**Bold** and *italic*.

- A bulleted list
- Another item

1. A numbered list
2. Another item

[A link to somewhere](https://example.org)

> A quote or callout.
```

The preview shows exactly how it will render. If it looks right in the preview, it will look right on the live site.

Sending to the publisher

The publisher for PATHFINDER content receives files at:

`info@the-pathfinder-project.eu` (confirm with your coordinator — this may change)

To send:

1. Open a new email to the publisher.
2. Subject line: something like "News item: Sarajevo recap" or "Event: Sofia Forum".
3. Attach the `.md` file you downloaded.
4. Optionally include a note if there's context or a suggested publish time.
5. Send.

The publisher will upload the file to GitHub, at which point the site will update within about a minute.

FAQs

"Can I edit an already-published post?" — The composer only creates new files. To edit an existing news item or event, ask the publisher (they can do it directly on GitHub) or see [editing-directly.md](#) if you're comfortable with the web editor.

"What if I don't upload an image?" — Then the PATHFINDER logo appears on the card and article. Perfectly fine as a default.

"What if I want to save my work and come back later?" — Click Download. Keep the `.md` file on your computer. Later, open it in any text editor, edit, and re-attach when you're ready.

"Is my content saved anywhere while I'm typing?" — No. If you close the tab before downloading, your work is lost. Download before you close.

"Can I preview my post on the live site before it publishes?" — Only if your publisher opens a Pull Request (draft mode) instead of committing directly. Ask them to do that if you want a private preview URL before it goes live.

"Someone else already saw the composer URL. Is that a security problem?" — No. The composer generates text; it can't publish anything. Only committed changes on GitHub reach the site, and only your maintainer can commit. See [troubleshooting.md](#) for more on this.

If something in the composer doesn't work as expected, check [troubleshooting.md](#) or contact the technical maintainer.

Publishing content

This guide is for the **publisher** — the person who takes content submitted by partners (via the composer or by email) and commits it to the GitHub repository. Once committed, the site rebuilds and updates within about one minute.

If you've never used GitHub's web editor before, don't worry — this guide walks through it in full. It's about ~5 clicks per publish once you're used to it.

What you'll need

- A GitHub account.
- **Write access to the PATHFINDER repository** — your technical maintainer adds you. Confirm you can see an "Add file" button when browsing the repo.
- A modern browser (Chrome, Firefox, Edge, Safari).

The 60-second version

1. Someone sends you a `.md` file (or an email with the text).
2. Go to `github.com/<org>/pathfinder/tree/main/src/content/news` (for news) or `.../events` (for events).
3. Click **Add file** → **Upload files**, drag the `.md` file, commit.
4. Wait ~1 minute. The site updates.

That's it. The rest of this doc explains the details, alternatives, and edge cases.

Publishing a file that came from the Composer

Coordinators using the [Composer](#) send you a `.md` file. Here's the full flow:

1. Save the attached `.md` file from the email to your computer.
2. Open the file in any text editor and skim the content. Check:
 - **Sender identity**: does this email come from the person it claims to? A quick reply asking "did you send this?" is fine if you're unsure. Standard email hygiene — the same as accepting any editable content from anyone.
 - **Facts are right**: dates make sense, no obvious typos, tone matches the project.
 - **Image referenced but not attached?** — the file might mention `/images/news/foo.jpg`. If that image doesn't exist in the repo yet, either ask for it or upload it first (see [managing-images.md](#)).
3. Go to GitHub, navigate to the right folder:

- News → `github.com/<org>/pathfinder/tree/main/src/content/news`
- Events → `github.com/<org>/pathfinder/tree/main/src/content/events`

4. Click **Add file** → **Upload files**.

5. Drag the `.md` file into the drop zone (or click "choose your files").

6. Scroll down. Under **Commit changes**:

- Leave the default commit message, or type something short like "Add news: Sarajevo recap".
- Choose **Commit directly to** `main` (unless your team's rule is to use pull requests — see below).

7. Click **Commit changes**.

8. Watch the **Actions** tab (top of the repo) — you'll see a new workflow run kick off. When it turns green (~1 minute), the site is updated. If it turns red, see [troubleshooting.md](#).

Publishing content that arrived as pasted text (not a file)

If someone sent you the content in the email body instead of as an attachment:

1. Copy the entire text (starting with `---` at the top).
2. Go to the right folder on GitHub (`src/content/news` or `src/content/events`).
3. Click **Add file** → **Create new file**.
4. In the filename box at the top, type a name like `2027-05-15-short-slug.md` (see the naming rules below).
5. Paste the entire text into the file body.
6. Scroll down, commit directly to `main` .

Filename rules

Files in `src/content/news/` and `src/content/events/` follow this pattern:

```
YYYY-MM-DD-short-slug.md
```

- `YYYY-MM-DD` — the date of the news item (for events, use the start date).
- `short-slug` — a short, lowercase, hyphenated version of the title. No spaces, no punctuation. Example: `sarajevo-dialogue-recap` .
- The file extension must be `.md` .

Files downloaded from the Composer already have valid names — you can commit them as-is.

Editing an already-published post

To fix a typo or update information after publishing:

1. Navigate to the file on GitHub (e.g. `src/content/news/2027-05-15-sarajevo-recap.md`).
2. Click the pencil icon (top right of the file view).
3. Make your edits.
4. Scroll down, add a short commit message ("Fix date in Sarajevo recap"), commit.
5. Site rebuilds within a minute.

Deleting a post

If a post shouldn't be public any more:

1. Navigate to the file on GitHub.
2. Click the trash icon (top right).
3. Confirm and commit.
4. Alternatively, keep the file but add `draft: true` to its frontmatter — the post disappears from the site but stays in the repo for later.

Using pull requests for review (optional, safer)

If your team's rule is to review changes before they go live:

1. When you commit (step 6 above), choose **Create a new branch** instead of "Commit directly to `main`". Name the branch something like `add-sarajevo-recap`.
2. Click **Commit changes** — this creates a Pull Request.
3. GitHub Actions runs the build against your branch. When it finishes, you get a preview URL in the PR comment (if configured).
4. Have a colleague review the PR.
5. Merge the PR into `main`. The site updates within a minute of the merge.

Branch protection can enforce this — if your maintainer has enabled it, you **have to** use a PR for any change to `main`. That's a safety net, not an obstacle.

Common issues while publishing

Red X on the commit / a failed deploy → the file is malformed (usually a YAML syntax error). Open the Action log; it names the offending file and field. Fix it with another commit. See [troubleshooting.md](#) for common causes.

"Author does not have write access" → your GitHub account isn't a repo collaborator yet. Ask the technical maintainer to invite you.

Two people committing at once → GitHub handles this fine, but if you see a merge conflict, it usually means two people edited the same file. Pull the latest, resolve, commit again.

Full field references for what goes in a news or event file are in [editing-directly.md](#).

Editing directly on GitHub

This guide covers editing the site's content **without the Composer** — useful for the publisher, or for coordinators comfortable with GitHub's web editor. It covers news, events, partners, Advisory Board members, and outputs.

If you'd rather use a form-based interface for news and events, see [composer.md](#).

What's a content file?

Every piece of content on the site is a plain text file in `src/content/`. Files use Markdown format with a small YAML block at the top called "frontmatter":

```
---
title: My news item
date: 2027-05-15
summary: A short preview.
---

The full article in Markdown goes down here.
```

The `---` lines are required. Everything between them is frontmatter (structured fields). Everything below is the body content.

Content folders

Content type	Folder	One file per...
News	<code>src/content/news/</code>	News item
Events	<code>src/content/events/</code>	Event
Partners	<code>src/content/partners/</code>	Partner organisation
Advisory Board	<code>src/content/advisory-board/</code>	AB member
Featured outputs	<code>src/content/outputs/</code>	Deliverable getting its own detail page

Adding a news item

File location: `src/content/news/`

Filename: `YYYY-MM-DD-short-slug.md` (e.g. `2027-06-15-sarajevo-recap.md`).

Template (in `src/content/news/_template.md`):

```
---
title: Headline for the news item
date: 2027-06-15
summary: One or two sentences (at least 20 characters).
author: Coordinator name          # optional
image:                             # optional
  src: /images/news/photo.jpg
  alt: A short description of the photo
  credit: Photographer on Source
tags:                               # optional
  - event-recap
draft: false                        # set to true to hide while editing
---

Write the article in Markdown here.

## Use headings, lists, and links freely
```

Required fields: `title`, `date`, `summary`. Everything else is optional. If no `image` is provided, the PATHFINDER logo shows as the default on the card and article.

Adding an event

File location: `src/content/events/`

Filename: `YYYY-MM-DD-short-slug.md` (use the start date).

Template:

```
---
title: Short event title
startDate: 2027-03-15
endDate: 2027-03-16                # optional, omit for single-day events
location: City, Country
country: Greece                     # must be one of the allowed values (see below)
format: In-person                   # must be one of: In-person, Online, Hybrid
summary: One or two sentences (at least 20 characters).
image:                             # optional
  src: /images/events/photo.jpg
  alt: A short description of the photo
  credit: Photographer on Source
registrationUrl: https://...       # optional
draft: false                       # set to true to hide while editing
---

Full event description in Markdown here.
```

Allowed values for `country`: Slovakia , Greece , Bosnia & Herzegovina , Bulgaria , Serbia , Italy , United Kingdom , European Union , Online .

Allowed values for `format`: In-person , Online , Hybrid .

Typos in these fields will fail the build with a clear error.

Editing a partner

Each partner has one file in `src/content/partners/` . Fields:

```
---
name: Full official name
shortName: Short name / acronym
country: Slovakia                # same allowed values as events
role: Coordinator                # or "Beneficiary"
website: https://example.org    # optional
logo: /images/partners/foo.png  # optional
summary: One-liner about the org.
contribution: What they do in PATHFINDER.
team:                           # list of team members
  - name: Denisa Karabová
    role: Project coordinator
    bio: Optional short bio.
    photo: /images/team/denisa-karabova.jpg # optional
  - name: Ali Honaramiz
    role: European Dialogue team
order: 1                        # order the partner appears (lower first)
---

Optional paragraph of extra body text (rarely used).
```

To add or remove team members, just edit the `team:` list. Each entry needs `name` and `role` ; `bio` and `photo` are optional. If `photo` is omitted, an initials avatar appears in brand teal.

Editing an Advisory Board member

Each member has one file in `src/content/advisory-board/` :

```

---
name: Full name
country: Italy # same allowed values as events
affiliation: Institution / employer
expertise: One-line description of expertise areas
bio: One or two sentences about them and their PATHFINDER role.
photo: /images/team/name.jpg # optional
order: 1 # display order on the page
---

```

Editing a featured output

Only the **three flagship outputs** get their own detail pages. Files live in `src/content/outputs/`.

```

---
id: D6.1 # deliverable ID from the DoA
title: Report title
workPackage: WP6 # for internal reference only, not shown publicly
leadPartner: CREDI # for internal reference only, not shown publicly
dueMonth: 7 # project month
summary: One or two sentences shown on the flagship card.
featured: true # true to show on the outputs page
fileUrl: https://... # optional link to the published PDF
order: 1 # display order among flagships
---

Optional body text (shown on the detail page under the summary).

```

Editing the About page, Home page, or Consortium intro

These pages are NOT content-file-driven — they live in `src/pages/`. They're Astro files (a mix of HTML and expressions). Coordinators can still edit the text on them by finding the exact string in the file and changing it — just be careful not to break the tags (angle brackets and curly braces).

Files:

- Home page copy: `src/pages/index.astro`
- About page: `src/pages/about.astro`
- Consortium intro (above the partner sections): `src/pages/consortium.astro`

Site-wide facts (project name, tagline, description, coordinator info, contact email, country list) live in `src/data/site.ts`. One place to change them all.

Hiding a post without deleting it

Every content file supports a `draft: true` field in the frontmatter. When set, the file is skipped by the site build — it stays in the repository but doesn't appear on the live site. Useful for:

- Drafting a post over several days.
- Temporarily unpublishing a post without losing the file.
- Test posts you don't want visitors to see.

Common YAML gotchas

- **Indentation** — always exactly 2 spaces. Tabs will break the build.
- **Dates** — always `YYYY-MM-DD`. Don't use slashes or written dates.
- **Colons in titles or summaries** — wrap the whole value in double quotes: `title: "Learning: what we found"`.
- **Line breaks in fields** — avoid. Keep field values on a single line. Multi-line content belongs in the body (below the second `---`).

The build tells you what's wrong

If you commit a file with a broken schema, GitHub Actions fails and the site doesn't update. Open the Action log — it names the file and the exact problem. See [troubleshooting.md](#).

For image uploads and Creative Commons sources, see [managing-images.md](#).

Managing images

The PATHFINDER site uses images in several places: hero photos, news article headers, event photos, partner logos, and team member portraits. This guide covers where they live, how to add them, and where to find Creative Commons ones.

Where images live

Images go in `public/images/`, in one of these subfolders:

Subfolder	Used for	Referenced in Markdown as
<code>public/images/hero/</code>	Home page hero photo	<code>/images/hero/filename.jpg</code>
<code>public/images/news/</code>	News article images	<code>/images/news/filename.jpg</code>
<code>public/images/events/</code>	Event photos	<code>/images/events/filename.jpg</code>
<code>public/images/team/</code>	Partner + Advisory Board portraits	<code>/images/team/filename.jpg</code>
<code>public/images/partners/</code>	Partner organisation logos	<code>/images/partners/filename.png</code>

Note that the path in Markdown starts with `/images/...`, NOT `/public/images/...`. The `public/` folder is dropped at build time.

Adding an image on GitHub

1. Navigate to the right subfolder in the repo, e.g.
`github.com/<org>/pathfinder/tree/main/public/images/news`.
2. Click **Add file** → **Upload files**.
3. Drag your image into the drop zone.
4. Commit.

The image is now at `https://<your-site>/images/news/yourfile.jpg` and can be referenced in any content file.

Referencing an image in content

In news or event frontmatter:

```
image:
  src: /images/news/your-file.jpg
  alt: A short description of what the photo shows
  credit: Photographer Name on Unsplash
```

For team member portraits (in partner or Advisory Board files):

```
photo: /images/team/denisa-karabova.jpg
```

Image requirements

- **License:** Creative Commons (or public domain, or your own photo). See sources below.
- **Format:** JPEG (`.jpg`) for photos, PNG for logos, WEBP for smaller file sizes.
- **Size:**
 - Hero and article images: aim for **1600×900** or similar landscape ratio. Max 500 KB.
 - Team portraits: **400×400** square. Max 100 KB.
 - Partner logos: transparent PNG, roughly square, max 200 KB.
- **Naming:** lowercase, hyphens-not-spaces, descriptive: `sarajevo-workshop-2027.jpg` .

Use a compression tool if your images are too big:

- [Squoosh](#) — drag-and-drop, in your browser.
- Or any image editor's "Export for web".

Alt text (accessibility)

Every image on the site needs `alt` text. Not decorative alt like "photo" — a real description of what's in the image:

- ❌ Bad: `alt: photo`
- ❌ Bad: `alt: PATHFINDER event`
- ✅ Good: `alt: Twelve young researchers seated in a circle discussing AI ethics at the Sarajevo workshop`

If the image is purely decorative (e.g. a background pattern), leave alt empty (`alt: ""`) or omit the field entirely.

Where to find Creative Commons images

For general photos (workshops, people, conference-style scenes):

- [Unsplash](#) — Free for any use, attribution appreciated but not required. Enormous library. Great for hero images and article photos.
- [Pexels](#) — Similar to Unsplash. Free for any use.
- [Wikimedia Commons](#) — Filter by license (CC BY or CC BY-SA). Wide variety, includes historical and location photos. Attribution required — always record the credit.

For EU-specific imagery:

- [EU Audiovisual Service](#) — Official EU photos and videos. Attribution required, formatted as "© European Union, [year]".

For portraits of team members and AB members:

- Ideally use photos the person supplies themselves. Ask them.
- If they have a professional profile page (university, LinkedIn), often you can request permission to use their photo from there.
- Don't scrape social media portraits without permission.

Recording the credit

For images with attribution requirements, always fill in the `credit:` field:

```
image:  
  src: /images/news/team-workshop.jpg  
  alt: Workshop participants gathered around a table  
  credit: Fauxels on Pexels
```

For EU images:

```
credit: © European Union, 2027 / Photographer Name
```

Team portraits don't currently render a visible credit (it's assumed you got permission). Keep a record of where each photo came from in a private log.

The default logo fallback

If a news item or event has no `image:` in its frontmatter, the site automatically shows the **PATHFINDER logo** as the hero image, on a warm surface background. This looks intentional — you don't need to add an image for every post.

Skip the image field when:

- You don't have a photo yet.
- The post is short-form (an announcement, a link).
- The photo would be generic filler (a stock keyboard, a generic office).

Replacing the hero photo on the Home page

The Home page hero currently uses `/images/hero/youth-collaboration.jpg` (a placeholder from Unsplash). To swap it:

1. Upload the new photo to `public/images/hero/`.
 2. Open `src/pages/index.astro`.
 3. Find the line `src="/images/hero/youth-collaboration.jpg"` and change the filename.
 4. Also update the small `Photo via Unsplash` credit at the bottom-right of the hero if the credit changes.
-

Broken image on the site? See [troubleshooting.md](#).

Troubleshooting

Common issues and how to fix them. Symptoms first, causes and fixes below.

Symptoms

- [A commit failed — red X on the commit / broken deploy](#)
 - [The site says "Page not found" for a page I just added](#)
 - [An image doesn't show up on the live site](#)
 - [The Composer preview doesn't update when I type](#)
 - [The Composer's "Download .md" button does nothing](#)
 - [I published something and want to undo it](#)
 - [My local `npm run dev` shows 404 on a page I edited](#)
 - [Someone I don't recognise is submitting content to the publisher](#)
-

A commit failed

Symptom: You commit a file (news, event, partner, etc.) and get a red X mark instead of a green check. Live site doesn't update.

Cause: The file has a formatting error — usually a YAML frontmatter issue.

Fix:

1. Click the red X on the commit to see the failed GitHub Action.
2. Click into the failed job. Scroll down to find the error message.
3. Common error patterns:
 - **"Missing required field 'title'"** → you forgot the `title:` line in the frontmatter.
 - **"Invalid enum value 'Greek' for 'country'"** → the country name doesn't match the allowed list. Fix to `Greece` (see [editing-directly.md](#) for allowed values).
 - **"Invalid date"** → your `date:` field isn't in `YYYY-MM-DD` format.
 - **"Invalid url"** → a URL field is missing the `https://` prefix.
 - **"expected ':' but found something else"** → YAML syntax error, usually a wrong indent (must be 2 spaces, no tabs) or missing colon.
4. Edit the file to fix the issue, commit again. The build re-runs automatically.

The site says "Page not found"

Symptom: You commit a new news item or event, GitHub Actions turns green, but visiting the URL shows the 404 page.

Cause 1: You edited a file's slug or date after publishing, changing the URL.

- **Fix:** check the file's filename. The URL is derived from it (e.g. `2027-05-15-sarajevo-recap.md` → `/news-events/news/2027-05-15-sarajevo-recap`).

Cause 2: The file has `draft: true` in its frontmatter.

- **Fix:** change to `draft: false` (or delete the line — false is the default), commit.

Cause 3: Browser or CDN cache.

- **Fix:** hard refresh (Cmd+Shift+R on Mac, Ctrl+Shift+R on Windows).

An image doesn't show

Symptom: You referenced an image in a content file but a broken-image icon appears on the site.

Cause 1: The image file isn't actually uploaded to `public/images/...` in the repo.

- **Fix:** check whether the file exists in the correct subfolder on GitHub.

Cause 2: Wrong path in the frontmatter.

- **Fix:** the path must start with `/images/...`, NOT `/public/images/...`. The `public/` prefix is dropped at build time.

Cause 3: Filename mismatch (case-sensitivity).

- **Fix:** `Sarajevo.jpg` and `sarajevo.jpg` are different files. Filenames on the live site are case-sensitive.

The Composer preview doesn't update when I type

Symptom: You're testing the composer locally (`npm run dev`). Typing in fields doesn't change the preview panel. Buttons don't respond.

Cause: Vite's dependency cache has gone stale (usually happens right after installing a new package).

Fix (in the terminal running `npm run dev`):

```
# Stop the dev server (Ctrl+C in the terminal running it)
rm -rf node_modules/.vite
npm run dev
```

This clears Vite's pre-bundled dependency cache. First page load after this will be slightly slower while it re-optimizes.

On the **live site** (after deploy), this issue does not exist — the production build bundles everything up-front.

The Composer's Download button does nothing

Symptom: Clicking "↓ Download .md" doesn't save a file.

Cause 1: Your browser is blocking the download.

- **Fix:** check the browser's download settings. Some strict configurations block automatic downloads from unknown pages.

Cause 2: Same Vite dep cache issue as above.

- **Fix:** same as above — restart the dev server after clearing `node_modules/.vite`. (Only applies to local dev; the live site is fine.)

Undoing a published change

If a commit went wrong and you want to revert it:

Option 1 (via GitHub UI):

1. Go to the repo's **Commits** page (`github.com/<org>/pathfinder/commits/main`).
2. Find the bad commit.
3. Click the "... " menu on that commit → **Revert**.
4. Commit the revert. Site rebuilds within a minute, back to the pre-bad-commit state.

Option 2 (delete the file):

If a commit added a bad file, just delete the file:

1. Go to the file on GitHub.
2. Click the trash icon (top right).
3. Confirm and commit. Site rebuilds without that content.

Local `npm run dev` shows 404

Symptom: You edited a content file's frontmatter (added or removed fields). The dev server suddenly shows 404 for that page.

Cause: Astro's content collection cache in the running dev server got stale.

Fix: restart the dev server. Ctrl+C, then `npm run dev` again.

This does not affect the production build — only the running dev server.

Someone I don't recognise is submitting content

Symptom: The publisher receives a `.md` file from an email address they don't recognise.

Reason to be careful: the composer page is public (unlisted URL, but findable). Anyone can generate a valid-looking Markdown file. Publishing it without verification would put someone else's content on the site.

Fix: standard email hygiene. Ask the coordinator or the presumed sender to confirm they sent it. Don't publish unverified content — the same way you wouldn't act on any unverified email request.

The composer itself is inert (see the security explanation in the composer page's info panel). The publishing step is the security perimeter, and it's a human decision.

If your issue isn't covered here, contact the technical maintainer or open an issue on the repo.

Deployment and operations

This guide is for the **technical maintainer** — one person who's comfortable with GitHub, the command line, and DNS. It covers one-time setup, ongoing operations, and security posture.

One-time setup (before first launch)

1. Create the GitHub repo

Create a repo named `pathfinder` (or whatever you prefer) in the organisation that will own the project. Public is fine (the content is meant to be public anyway); private also works if you're on Pro / Team / Enterprise.

2. Push the initial commit

From your local machine:

```
cd ~/Documents/pathfinder
git init
git add .
git commit -m "Initial site"
git remote add origin git@github.com:<your-org>/pathfinder.git
git branch -M main
git push -u origin main
```

3. Configure the site URL

Open `astro.config.mjs`. Change the `SITE` constant to your live URL:

```
const SITE = "https://pathfinder-project.eu";           // custom domain
// or
const SITE = "https://<your-org>.github.io";           // github.io user/org page
```

If you're using a project page (URL includes `/pathfinder`), also set `BASE`:

```
const BASE = "/pathfinder";
```

Commit and push.

4. Enable GitHub Pages

On github.com, in your repo:

1. **Settings** → **Pages**.

2. **Source:** GitHub Actions.

That's it — the workflow in `.github/workflows/deploy.yml` runs on every push to `main` and deploys automatically.

5. (Optional) Set up a custom domain

If you own a domain like `pathfinder-project.eu`:

1. Create a file `public/CNAME` containing just the domain (no `https://`, no trailing slash):

```
pathfinder-project.eu
```

2. Commit and push.

3. At your DNS provider, add records per [GitHub's documentation](#):

- Apex domain: 4 A records pointing to GitHub's IPs.
- Subdomain (`www.pathfinder-project.eu`): CNAME to `<your-org>.github.io`.

4. In **Settings** → **Pages**, set the custom domain and enable **Enforce HTTPS** once the certificate is issued (~15 minutes).

6. Harden the repository

Still in **Settings**:

- **Branches** → **Add rule for** `main` :
 - Require a pull request before merging.
 - Require at least 1 approving review.
 - Require status checks to pass (`build`).
 - Do not allow force pushes.
- **Code security** → **Advanced Security**: enable Dependabot alerts.
- **Access** → **Manage access**: invite only the collaborators who need write access.
- **Organisation** → **Authentication security**: require 2FA for all collaborators.

7. Set up the publisher

Identify **one person** who will publish content (usually the project coordinator or communications officer). Add them as a repository collaborator with write access. Send them the link to [publishing.md](#).

If you don't want the publisher to have direct push access to `main`, add them as a collaborator with restricted permissions and require them to open PRs (branch protection above enforces this).

8. Set up a real mailbox

The site's contact and composer instructions point at `info@the-pathfinder-project.eu`. Make sure this actually exists and someone reads it. If you use a different address, update it in:

- `src/data/site.ts` — the `coordinator.email` field.
- `docs/composer.md` — the "Sending to the publisher" section.

9. Verify the first deploy

Push a small change (e.g. a comment update in the README), watch the Actions tab, wait for the green check, then visit the live URL. Walk all 5 top-navigation pages. Click into a news detail and an event detail. Try `/admin/compose`. Everything should render.

Ongoing operations

Monitoring deployments

- **Actions tab** — every push to `main` triggers a build + deploy. Watch here for failures.
- **Deployments panel** — the sidebar on the repo home shows recent deploys.
- Deploy time: ~1 minute end-to-end.

Adding a new dependency

If you install a new npm package:

```
npm install <package-name>
```

Test locally with `npm run dev` and `npm run build`. Commit both `package.json` and `package-lock.json`.

Common pitfall: while `npm run dev` is running, installing a new package can leave Vite's dep cache stale — the local site breaks. Fix: `rm -rf node_modules/.vite` then restart the dev server. Does not affect the production build.

Updating dependencies

Roughly monthly, or when Dependabot flags a vulnerability:

```
npm outdated          # see what's out of date
npm update             # patch + minor updates
```

For major version bumps (e.g. Astro 5 → Astro 6), read the changelog first and test carefully.

Local development

```
npm install          # once per fresh clone
npm run dev          # dev server at http://localhost:4321
npm run build        # build to dist/
npm run preview      # serve the built site locally
```

Requires Node 22+ (see [.nvmrc](#)).

Reverting a bad deploy

Content mistakes: see [troubleshooting.md](#).

Code mistakes: revert the commit via GitHub's "Revert" button (Commits page → three-dot menu → Revert). Or:

```
git revert <commit-hash>
git push
```

Redeploy happens automatically.

Security posture

The site's security model, in one paragraph:

The live site is static HTML/CSS/JS — visitors cannot write to anything. Only GitHub repository collaborators can push, and (with branch protection) every change goes through a reviewed pull request. GitHub Actions builds and deploys from every commit; the audit log records who did what. No third-party CDN dependencies at runtime — fonts are bundled from npm, images are self-hosted. The site collects no personal data and sets no cookies, so GDPR obligations are minimal (an "imprint" / legal notice may still be required depending on your jurisdiction).

The Composer page at [/admin/compose](#) is client-side only. It generates Markdown text and downloads it. It has no way to publish anything. Even a public URL with high traffic to this page creates no security exposure — the publishing step (a human on GitHub) is the security perimeter.

File structure

```
pathfinder/
├─ docs/                the documentation you're reading
├─ public/              static assets served at /
│  └─ images/           hero, news, events, partners, team, hero
│     └─ favicon.svg
├─ src/
│  └─ content.config.ts  Zod schemas – the source of truth for what's a valid content field
│     └─ content/        .md content files
│        └─ news/
│           └─ events/
│              └─ partners/
│                 └─ advisory-board/
│                    └─ outputs/
├─ data/                site-wide TS data (nav, project facts, objectives)
├─ components/           reusable .astro components
├─ layouts/              BaseLayout + ArticleLayout
├─ pages/                file-based routing
│  └─ admin/compose.astro the composer (unlisted)
├─ styles/global.css     Tailwind + design tokens
├─ .github/workflows/deploy.yml builds + deploys to GitHub Pages
├─ astro.config.mjs
├─ package.json
└─ README.md             for engineers – start here
```

Design tokens

Colours and fonts are CSS custom properties in `src/styles/global.css`, inside `@theme { ... }`. All values come from the official PATHFINDER visual identity.

Token	Value	Purpose
<code>--color-brand</code>	<code>#143B63</code>	Deep Navy — institutional
<code>--color-teal</code>	<code>#159E9C</code>	Innovation, secondary accent
<code>--color-accent</code>	<code>#FF8A30</code>	Bright Orange — energy, CTAs, eyebrows
<code>--color-surface</code>	<code>#F5F2EC</code>	Warm Light Background
<code>--color-ink</code>	<code>#2B2B2B</code>	Dark Graphite — text
<code>--font-display</code>	League Spartan	Headlines
<code>--font-sans</code>	Montserrat	Body

Change any of these to re-skin the site — the design system propagates.

For content editing (news, events, partners, etc.), see editing-directly.md.